Model Report: BERT-base-uncased + LoRA Fine-Tuning (Pretrained Model)

Milestone 2/4 requirement — "one pretrained model" + LoRA fine-tuning Project: Smart MCQ Solver Challenge

---

1. Motivation

With a TF-IDF baseline (MAP@3 = 0.280, below random) and a from-scratch Bi-LSTM Siamese Encoder (MAP@3 = 0.773) already built, this model fulfills the project's **pretrained model** requirement (Milestone 2) combined with the **LoRA fine-tuning** requirement (Milestone 4). The goal: leverage BERT's pretrained language representations, adapted efficiently via LoRA rather than full fine-tuning, and compare directly against the from-scratch model on the same leakage-free validation split.

---

2. Architecture

Base: `bert-base-uncased` via `transformers.AutoModelForMultipleChoice` — the standard Hugging Face architecture for exactly this task shape (used for SWAG/RACE-style benchmarks). Each question is expanded into 5 `[prompt, option]` pairs; BERT encodes each pair independently (shared weights across all 5), and a linear head scores each pair. A 5-way softmax across the scores gives the ranking used for MAP@3.

LoRA adaptation (via `peft`):

Parameter	Value
Rank (r)	8
Alpha	16
Dropout	0.1
Target modules	Attention `query`, `value` projections
Modules kept fully trainable	`classifier` (the multiple-choice head — freshly initialized, must be trained in full)

Only the LoRA adapter matrices and the classifier head are updated; the pretrained BERT weights themselves stay frozen throughout training.

---

3. Training Configuration

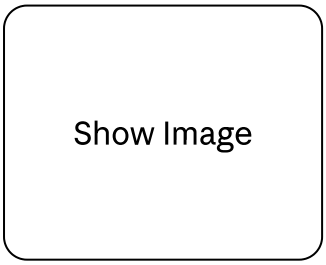
Hyperparameter	Value
Checkpoint	`bert-base-uncased`
Max sequence length	128 tokens
Learning rate	2e-4
Batch size (train/eval)	8 / 8
Epochs	8 (with early stopping, patience 3)
Weight decay	0.01
Optimizer	AdamW (via `Trainer`)
Metric for best checkpoint	Validation MAP@3
Random seed	42

Leakage-free split: same `StratifiedGroupKFold` grouped by core question (wrapper phrasing stripped) used for the Bi-LSTM model, so no near-duplicate question crosses the train/validation boundary. Validation set: 417 rows.

---

4. Results

Training curves



- Train loss decreases from ~3.25 to ~1.35 over 8 epochs (noisy but clearly trending down).
- Val loss decreases from ~3.2 to a plateau around ~2.6, with a widening gap versus train loss in later epochs — a mild overfitting signal, though less severe than the Bi-LSTM's.
- Val accuracy rises steadily from 0.31 (epoch 1) to a peak of ~**0.542** (epoch 7).
- Val MAP@3 rises from 0.48 (epoch 1) to a peak of ~**0.683** (epoch 7), then plateaus.

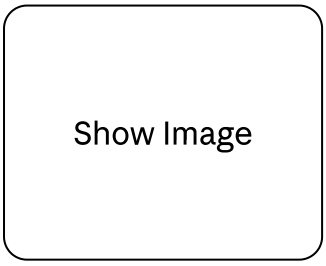
Final validation metrics (best checkpoint, epoch 7)

Metric	Score
Validation Accuracy (Top-1)	**0.542**
Validation MAP@3	**0.683**

Classification Report (Top-1, validation set, n=417)

Class	Precision	Recall	F1	Support
A	0.36	0.30	0.33	69
B	0.49	0.63	0.55	68
C	0.74	0.53	0.62	132
D	0.51	0.47	0.49	100
E	0.53	0.94	0.68	48
Accuracy			**0.54**	417
Macro avg	0.53	0.57	0.53	417
Weighted avg	0.56	0.54	0.54	417

Confusion Matrix



Notable per-class pattern: Class **E has the highest recall (0.94)** despite having the smallest support (48) — when the true answer is E, the model almost always finds it. Class **A is the weakest (recall 0.30, precision 0.36)** — A is frequently confused with B, C, and D roughly evenly (19, 11, 16 misclassifications respectively out of 69 true-A examples). Class **C has the highest precision (0.74)** but only moderate recall (0.53), meaning the model is conservative/confident when it does predict C, but still misses roughly half of true-C cases (spread across A, B, D, E).

5. Comparison Across All Models

Model	Validation MAP@3	Validation Accuracy (Top-1)
Random baseline	≈0.367	0.20
TF-IDF + Cosine Similarity	0.280	0.158
Bi-LSTM Siamese Encoder (from scratch)	**0.773**	0.713
BERT-base-uncased + LoRA (pretrained)	0.683	0.542

Notable finding: the from-scratch Bi-LSTM currently outperforms the LoRA-tuned BERT. This is worth stating plainly in the report rather than glossing over, since it runs counter to the usual expectation that pretrained transformers beat from-scratch models. Plausible contributing factors:

1. **LoRA's limited capacity at rank 8** may be too constrained to meaningfully adapt BERT's general-domain pretrained representations to this specific task (distinguishing near-paraphrase MCQ distractors) within only 8 epochs and ~1,600 training examples.
2. **The classifier head is the only fully-trainable component** besides the small LoRA matrices — with so few free parameters relative to a full fine-tune, the model may be undertrained for this specific decision boundary.
3. **Small dataset size (~1,600 rows after grouping)** may favor the Bi-LSTM's simpler, more direct architecture — parameter-efficient fine-tuning methods like LoRA typically show their advantage most clearly on larger datasets or when full fine-tuning would badly overfit; here, a lower-capacity from-scratch model apparently generalizes better on this exact split.
4. BERT-base's pretraining objective (masked language modeling, next-sentence-style tasks) may not transfer as directly to this competition's specific paraphrase-distractor structure as a dedicated architecture trained end-to-end on the task from the start.

This is a legitimate, useful result for the report's error-analysis section — it demonstrates the project's central lesson (from the TF-IDF failure onward) that **architecture fit to the specific data characteristics matters more than model size or pretraining alone.**

---

6. Limitations & Next Steps

- **Try full fine-tuning** (no LoRA) as an ablation, to isolate whether the gap versus the Bi-LSTM is due to LoRA's capacity constraint specifically, or something else.
- **Increase LoRA rank** (e.g. r=16 or r=32) and/or add more target modules (e.g. `key`, dense/output projections) to give the adapter more capacity.

- **Train for more epochs** with a lower learning rate — validation MAP@3 had only just plateaued at epoch 7–8, and mild overfitting was just beginning, suggesting there may be a small amount of headroom with better regularization (e.g. higher LoRA dropout, or a learning-rate schedule).
- **Try** `roberta-base` **or** `microsoft/deberta-v3-base` as alternate pretrained checkpoints — both are typically stronger than BERT-base on multiple-choice benchmarks.
- **Investigate the A-class weakness specifically** — check whether A-labeled questions share any systematic textual property (e.g. consistently the first/shortest option) that a stronger model could exploit, tying back to the earlier EDA finding on option length vs. correctness.

---

7. Why This Model Satisfies the "Pretrained Model" + LoRA Requirements

- Uses `bert-base-uncased`, a publicly pretrained checkpoint, as the encoder backbone.
- Fine-tuned via LoRA (`peft` library) rather than training from scratch — directly satisfying the project's Milestone 4 LoRA fine-tuning requirement.
- Evaluated on the same leakage-free validation protocol as the from-scratch model, enabling a fair, apples-to-apples comparison across all three required models.

---

8. Artifacts

File	Description
`05_bert_lora_finetune.py`	Full training + inference script
`bert_lora_model_archive.zip`	Saved LoRA adapter + tokenizer, zipped for download
`submission_bert_lora.csv`	Kaggle-ready submission, format-validated
`eda_outputs/bert_lora_training_curves.png`	Loss and metric curves across training
`eda_outputs/bert_lora_confusion_matrix.png`	Validation confusion matrix (top-1)
W&B run	Project `23f3003691-t22026`, run `BERT_LoRA_Finetune`

---

9. Next Steps for the Project

1. Build the **third model of choice** (e.g. `roberta-base` or `deberta-v3-base` fine-tune, or a

full-fine-tune ablation of BERT for direct comparison against the LoRA version).

2. Compare all models' W&B runs side-by-side (accuracy, macro F1, MAP@3) as required by the project's L1 viva checkpoint ("at least three runs compared using common metrics").
3. Consider **ensembling** the Bi-LSTM and BERT+LoRA predictions, since their error patterns may differ (worth checking whether they make the same or different mistakes on the same validation questions).